Conference New Perspectives in Evolution 2026, Schiermonikoog, the Netherlands

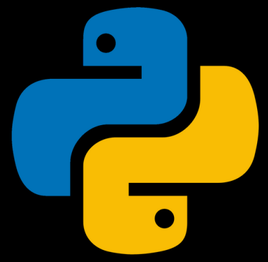
Test-driven development

IN ECOLOGY AND EVOLUTION

RAPHAËL SCHERRER

University of Groningen

university of groningen





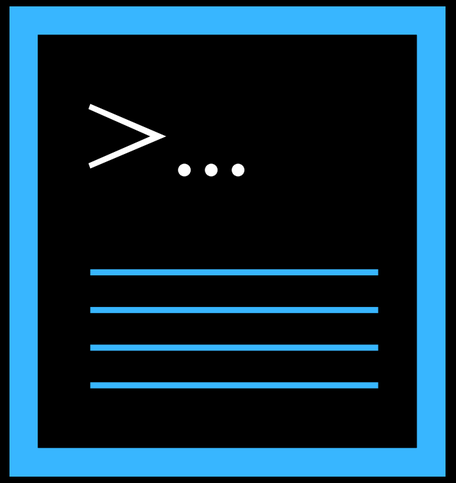


CODING

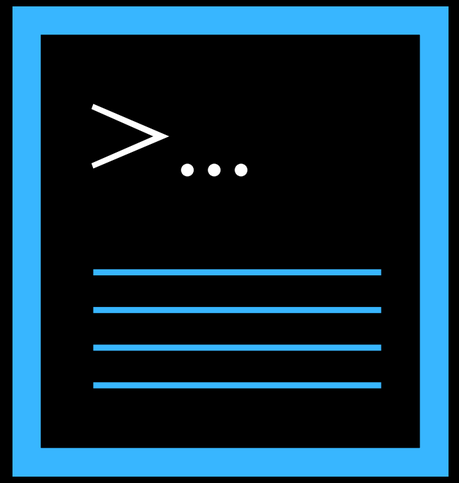








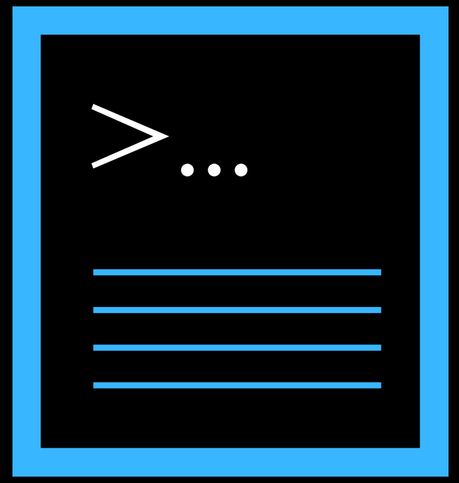
scripts, pipelines, analyses,
plotting, stats



scripts, pipelines, analyses,
plotting, stats

packages, functions, libraries,
frameworks





scripts, pipelines, analyses,
plotting, stats

packages, functions, libraries,
frameworks





simulations, toolboxes, personal
libraries











Gotta catch 'em all!™



BUG

a design defect in an engineered system - such as software [...] - that causes an undesired result

- Wikipedia

SYNTAX ERRORS

```
RBASIC Version 2.01
robotron  MKD  1989
25650 Bytes free
RBASIC Disk Version 2.00
Ok
10 SCREEN 2:COLOR 0,0: CLS
20 FOR I=20 TO 300 STEP 20
30 PSET(I,59),PSET(I,150),2
40 NEXT I
50 PAUSE
run
Syntax error in 30
Ok
```

EXCEPTIONS

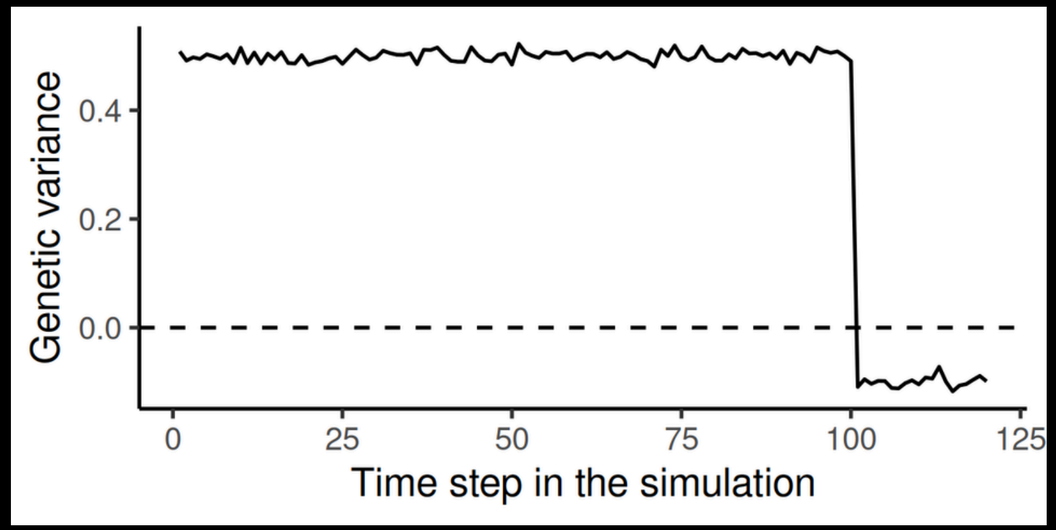
```
> rexp(-3)
Error in rexp(-3) : invalid arguments
```

EXCEPTIONS

```
> rexp(-3)
Error in rexp(-3) : invalid arguments
```

```
3  # stuff
4  # stuff
5  # stuff
6
7  if (input < 0) stop("invalid arguments")
8
9  # stuff
```

SILENT BUGS...




```
[1] "No. of individuals = 2 at time 1"
[1] "No. of individuals = 10 at time 2"
[1] "No. of individuals = 20 at time 3"
[1] "No. of individuals = 48 at time 4"
[1] "No. of individuals = 160 at time 5"
[1] "No. of individuals = 397 at time 6"
[1] "No. of individuals = 1133 at time 7"
[1] "No. of individuals = 2892 at time 8"
[1] "No. of individuals = 8090 at time 9"
[1] "No. of individuals = 22033 at time 10"
[1] "No. of individuals = 59933 at time 11"
[1] "No. of individuals = 163074 at time 12"
[1] "No. of individuals = 443251 at time 13"
[1] "No. of individuals = 1202048 at time 14"
[1] "No. of individuals = 3273127 at time 15"
[1] "No. of individuals = 8888617 at time 16"
```

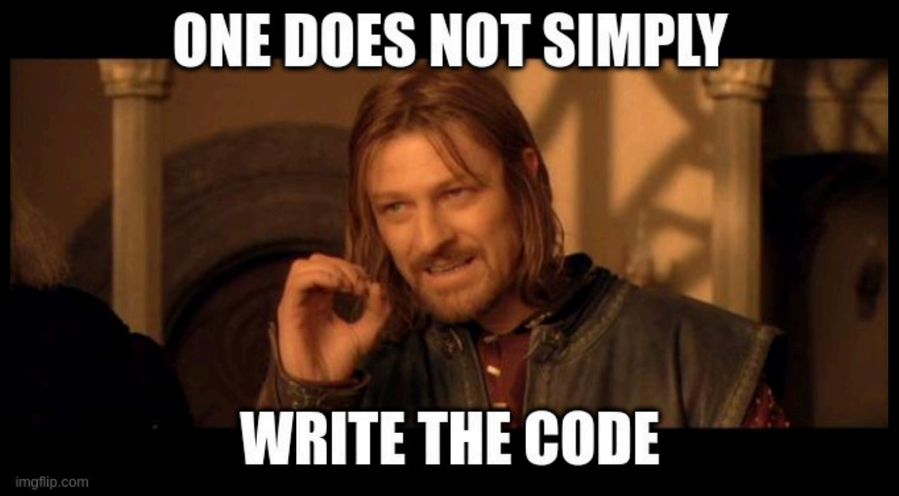
Details matter when modelling the effects of animal personality on the spatial distribution of foragers

Christoph Netz, Aparajitha Ramesh, Jakob Gismann, Pratik R. Gupte and Franz J. Weissing

Report of programming bugs in the DAISIE R package: consequences and correction

Luis Valente[1,2,*], Nadiah P. Kristensen[3], Albert B. Phillimore[4], Rampal S. Etienne[2]

TEST-DRIVEN DEVELOPMENT



Beck (2015) *Test-driven development by example*

```
# The code
is_prime = function(n) {
    ...
}
```

```
# The tests
if (is_prime(1) != TRUE) error()
if (is_prime(2) != TRUE) error()
if (is_prime(4) != FALSE) error()
if (is_prime(5) != TRUE) error()
```

```python
import random

def is_prime(n, k=10):
    # k = number of rounds (higher = more accuracy)

    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0:
        return False

    # write n-1 = d * 2^r
    r = 0
    d = n - 1
    while d % 2 == 0:
        d //= 2
        r += 1

    # witness loop
    for _ in range(k):
        a = random.randrange(2, n - 1)
        x = pow(a, d, n)  # fast modular exponentiation

        if x == 1 or x == n - 1:
            continue

        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False  # definitely composite

    return True  # probably prime
```



The tests

if (is_prime(1) != TRUE) error() ✓

if (is_prime(2) != TRUE) error() ✓

if (is_prime(4) != FALSE) error() ✗

if (is_prime(5) != TRUE) error() ✓

```
Test failed: is_prime(4) is not FALSE
>
```

```python
import random

def is_prime(n, k=10):
    # k = number of rounds (higher = more accuracy)

    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0:
        return False

    # write n-1 = d * 2^r
    r = 0
    d = n - 1
    while d % 2 == 0:
        d //= 2
        r += 1

    # witness loop
    for _ in range(k):
        a = random.randrange(2, n - 1)
        x = pow(a, d, n)  # fast modular exponentiation

        if x == 1 or x == n - 1:
            continue

        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False  # definitely composite

    return True  # probably prime
```



The tests

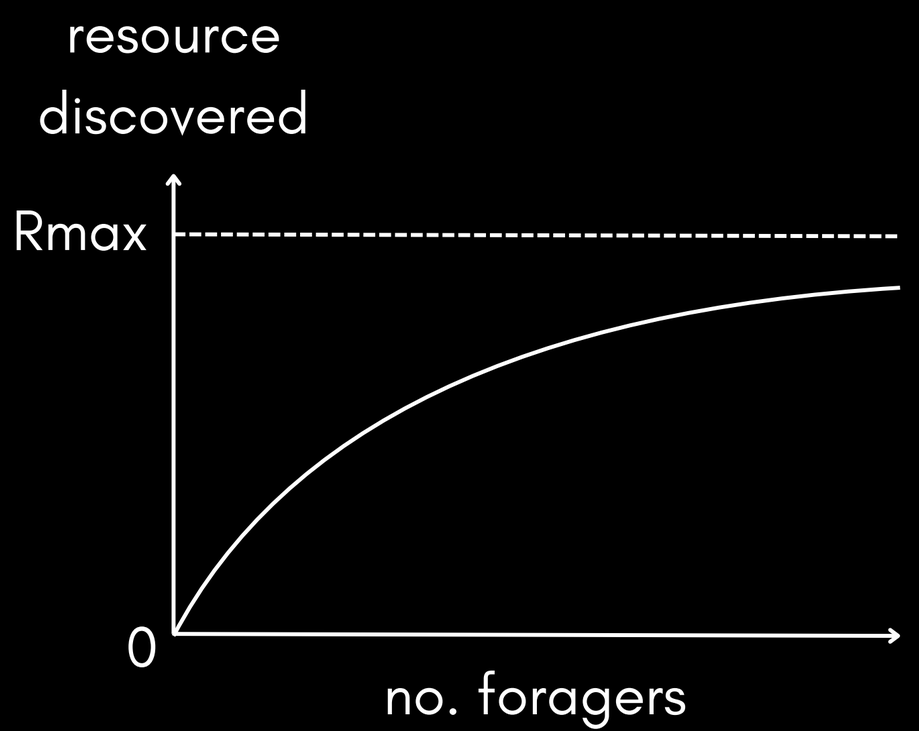
if (is_prime(1) != TRUE) error() ✓

if (is_prime(2) != TRUE) error() ✓

if (is_prime(4) != FALSE) error() ✓

if (is_prime(5) != TRUE) error() ✓

```
All tests pass yayy!
>
```

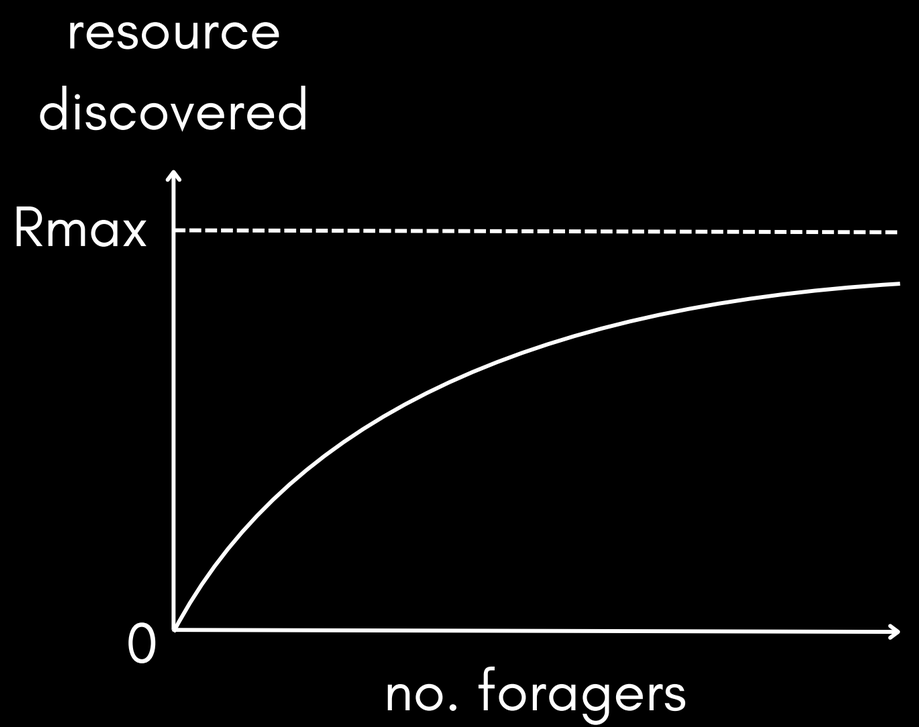

resource
discovered
Rmax
0
no. foragers

```
# Loop through time steps
while (t < tmax) {
    ...

    # Calculate resource discovered
    Rd = get_resources(Rmax, alpha, n)

    # Checks
    if (Rd < 0) error("Rd should be >= 0")
    if (Rd > Rmax) error("Rd should be <= Rmax")

    ...
}
```

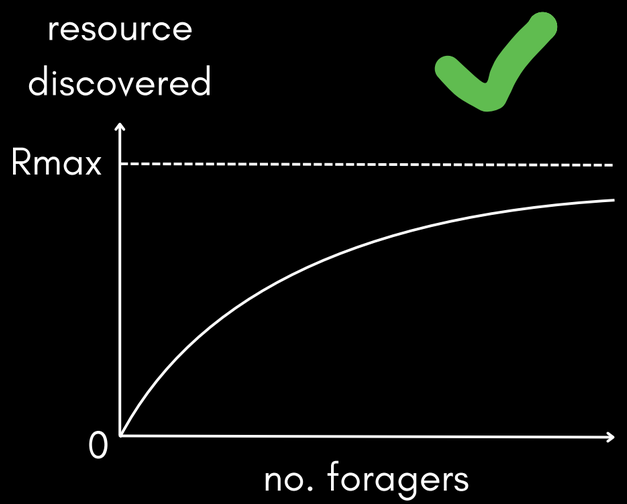


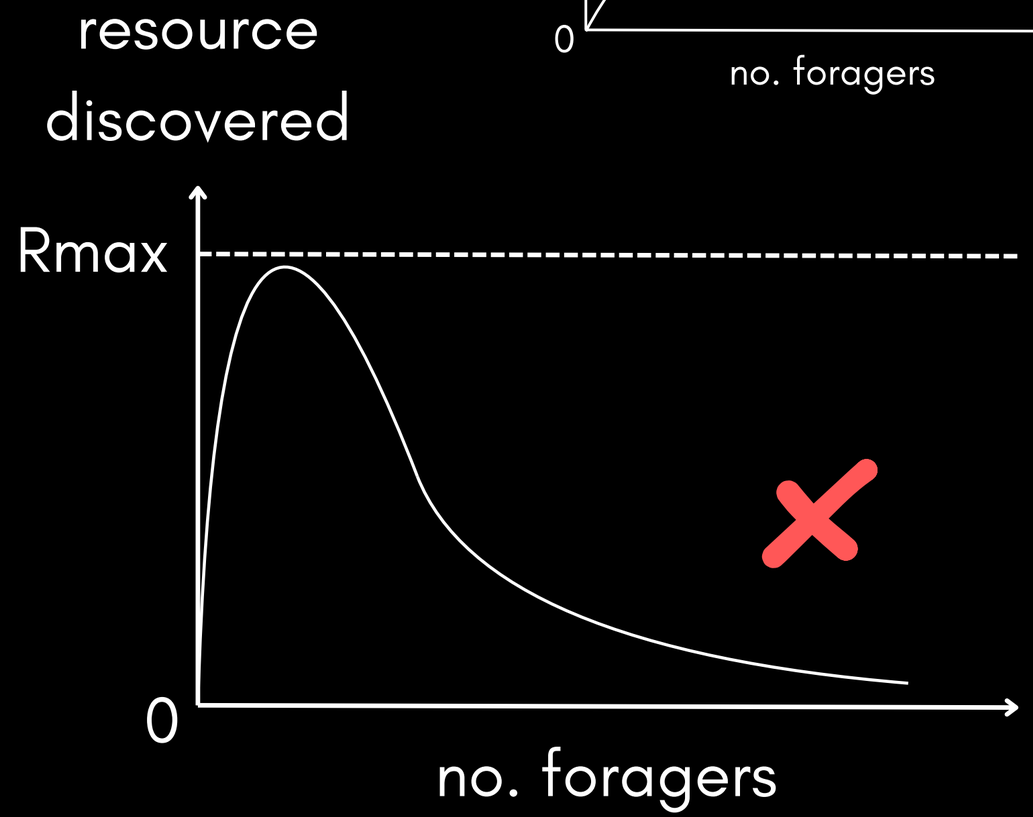
```
# Loop through time steps
while (t < tmax) {
    ...

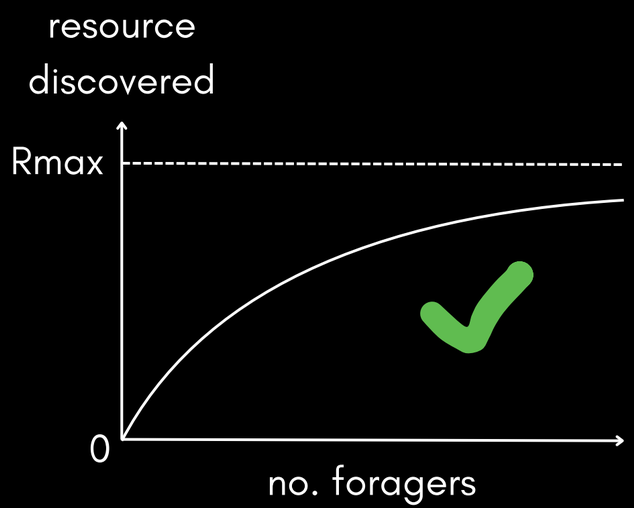
    # Calculate resource discovered
    Rd = get_resources(Rmax, alpha, n)

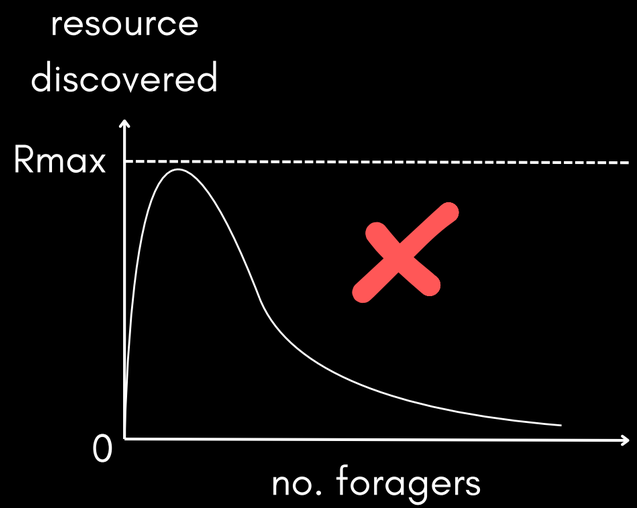
    # Checks
    if (Rd < 0) error("Rd should be >= 0")
    if (Rd > Rmax) error("Rd should be <= Rmax")

    ...
}
```










```
# The tests
r1 = get_resources(Rmax, alpha, 1)
r2 = get_resources(Rmax, alpha, 10)
r3 = get_resources(Rmax, alpha, 100)
if (r2 > r1) error()
if (r3 > r2) error()
```

FAQ

Will I ever be sure that my code is bug-free?



Will I ever be sure that my code is bug-free?

NASA's mission-ready software:

1 bug per **10,000** lines of code

Shi (2020) "Software Reliability in Space Applications — Facts, Trends and Challenges". https: //ntrs.nasa.gov/citations/20200000671

FAQ

Will I ever be sure that my code is bug-free?

NASA's mission-ready software:
1 bug per **10,000** lines of code



Shi (2020) "Software Reliability in Space Applications — Facts, Trends and Challenges". https: //ntrs.nasa.gov/citations/20200000671



Can't I just catch errors with assertions?

```
# Loop through time steps
while (t < tmax) {
    ...

    # Calculate resource discovered
    Rd = discover_resource(Rmax, alpha, n)

    # Checks
    if (Rd < 0) error("Rd should be >= 0")
    if (Rd > Rmax) error("Rd should be <= Rmax")

    ...
}
```



ASSERTIONS



Can't I just catch errors with assertions?

```
# Loop through time steps
while (t < tmax) {
    ...

    # Calculate resource discovered
    Rd = discover_resource(Rmax, alpha, n)

    # Checks
    if (Rd < 0) error("Rd should be >= 0")
    if (Rd > Rmax) error("Rd should be <= Rmax")

    ...
}
```

ASSERTIONS

TESTS

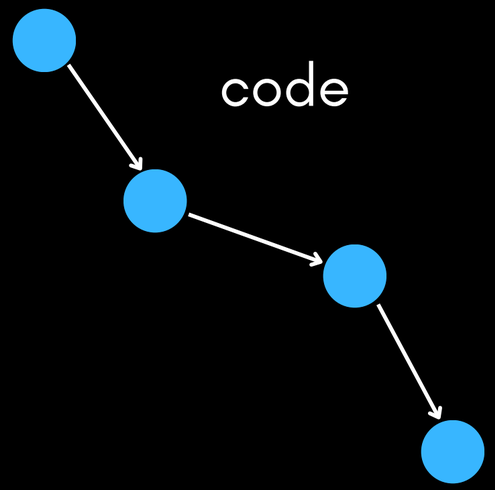
```
# The tests
r1 = get_resources(Rmax, alpha, 1)
r2 = get_resources(Rmax, alpha, 10)
r3 = get_resources(Rmax, alpha, 100)
if (r2 > r1) error()
if (r3 > r2) error()
```

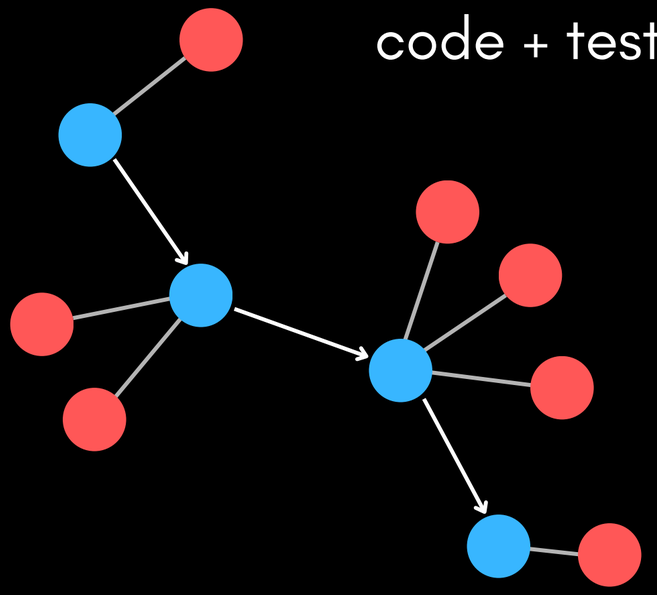
FAQ

Will it take me more time to code?

FAQ

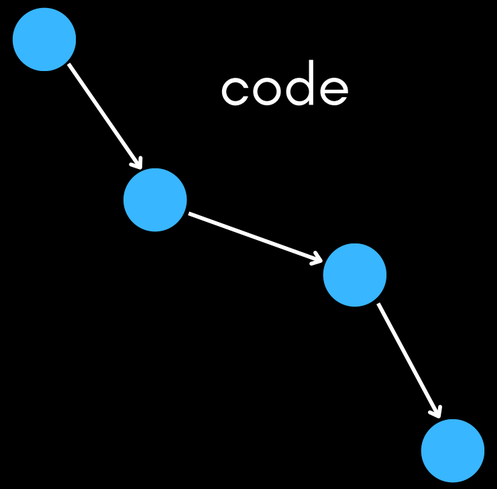
Will it take me more time to code?

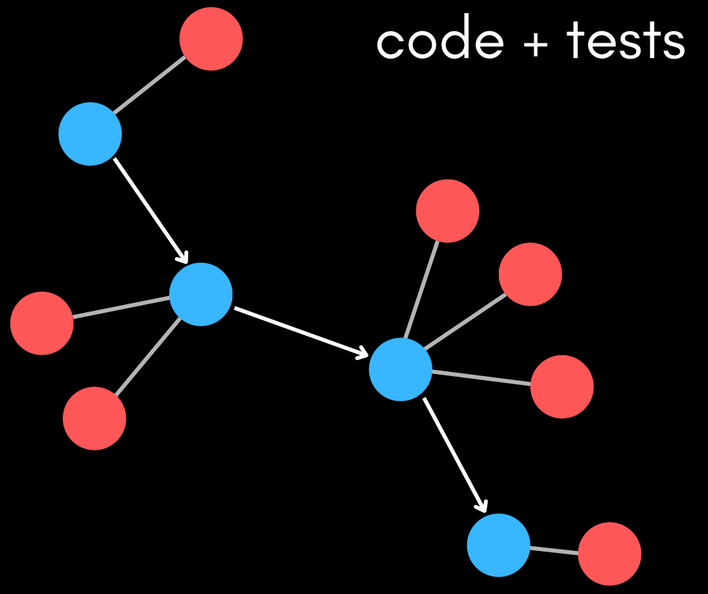




FAQ

Will it take me more time to code?



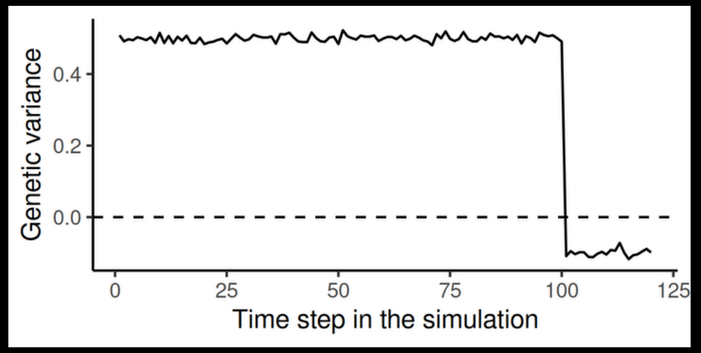


80% time spent debugging

Langr (2014) *Modern C++ programming with test-driven development: code better, sleep better*

FAQ

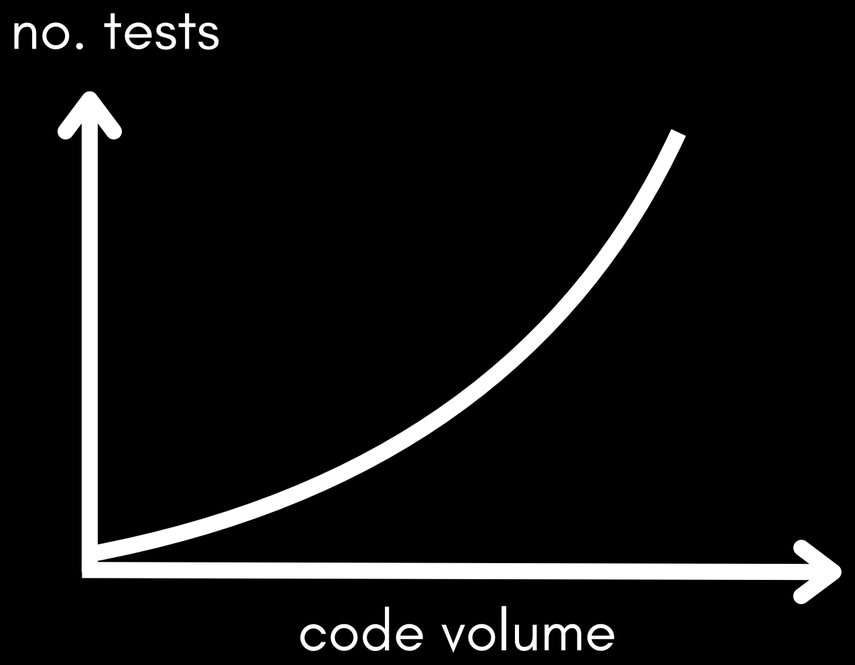
Can’t I just look at plots or intermediate output?

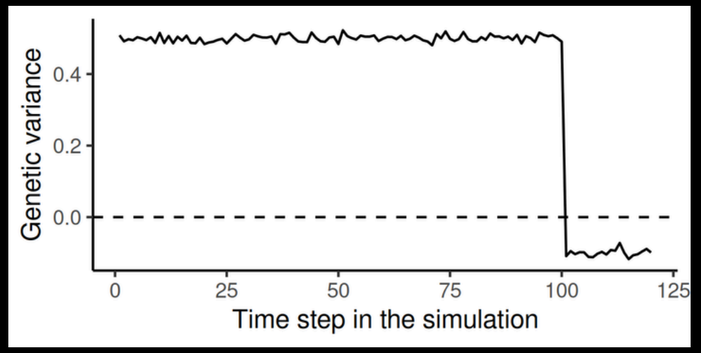



```
[1] "No. of individuals = 2 at time 1"
[1] "No. of individuals = 10 at time 2"
[1] "No. of individuals = 20 at time 3"
[1] "No. of individuals = 48 at time 4"
[1] "No. of individuals = 160 at time 5"
[1] "No. of individuals = 397 at time 6"
[1] "No. of individuals = 1133 at time 7"
[1] "No. of individuals = 2892 at time 8"
[1] "No. of individuals = 8090 at time 9"
[1] "No. of individuals = 22033 at time 10"
[1] "No. of individuals = 59933 at time 11"
[1] "No. of individuals = 163074 at time 12"
[1] "No. of individuals = 443251 at time 13"
[1] "No. of individuals = 1202048 at time 14"
[1] "No. of individuals = 3273127 at time 15"
[1] "No. of individuals = 8888617 at time 16"
```

FAQ

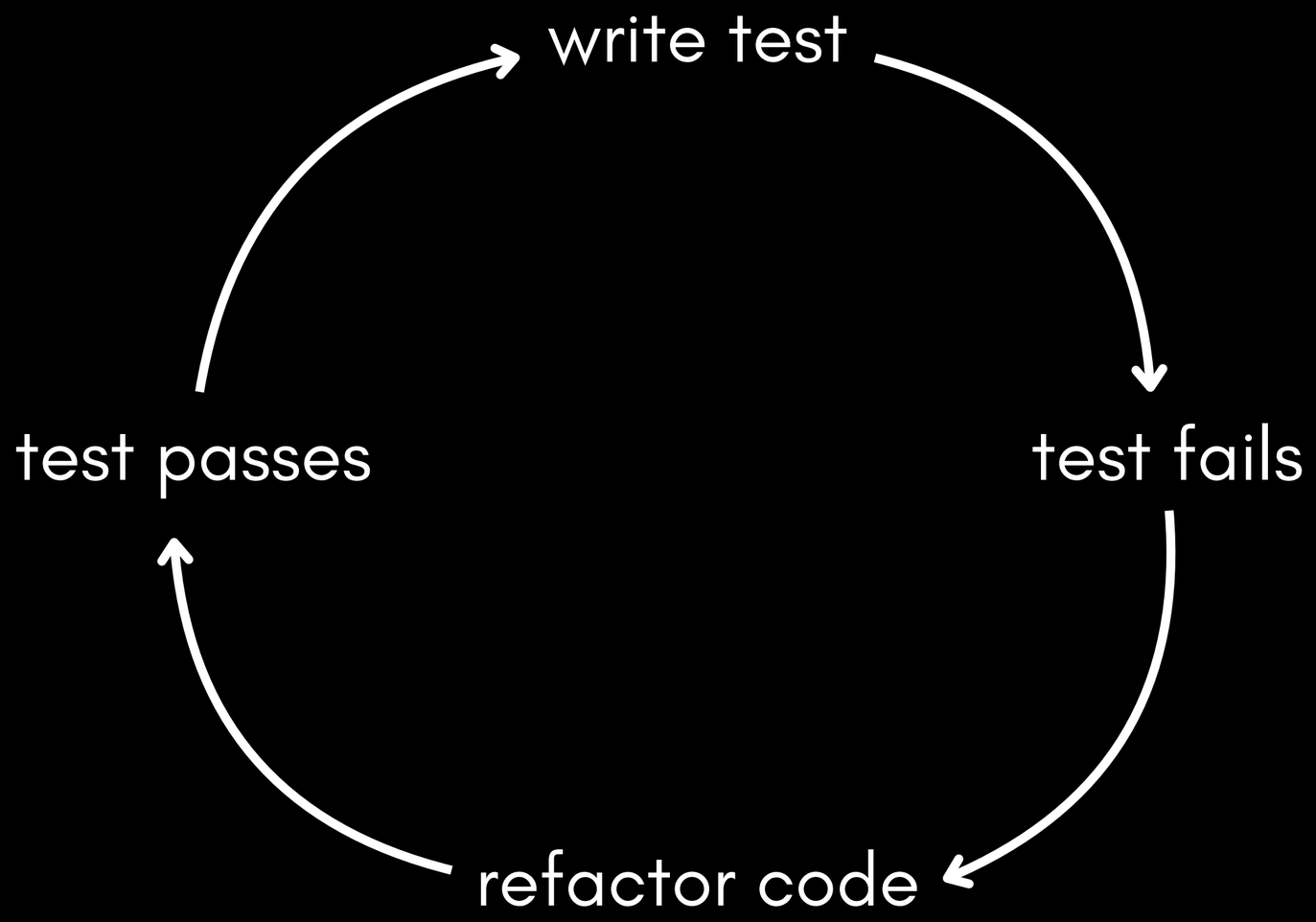
Can’t I just look at plots or intermediate output?



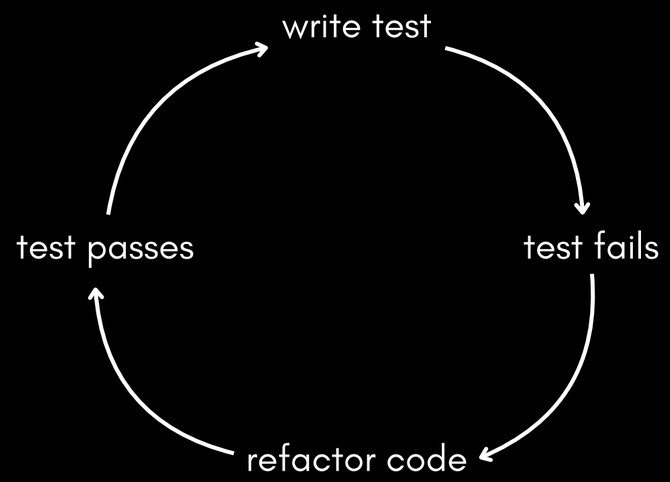



```
[1] "No. of individuals = 2 at time 1"
[1] "No. of individuals = 10 at time 2"
[1] "No. of individuals = 20 at time 3"
[1] "No. of individuals = 48 at time 4"
[1] "No. of individuals = 160 at time 5"
[1] "No. of individuals = 397 at time 6"
[1] "No. of individuals = 1133 at time 7"
[1] "No. of individuals = 2892 at time 8"
[1] "No. of individuals = 8090 at time 9"
[1] "No. of individuals = 22033 at time 10"
[1] "No. of individuals = 59933 at time 11"
[1] "No. of individuals = 163074 at time 12"
[1] "No. of individuals = 443251 at time 13"
[1] "No. of individuals = 1202048 at time 14"
[1] "No. of individuals = 3273127 at time 15"
[1] "No. of individuals = 8888617 at time 16"
```

REFACTORING CYCLE

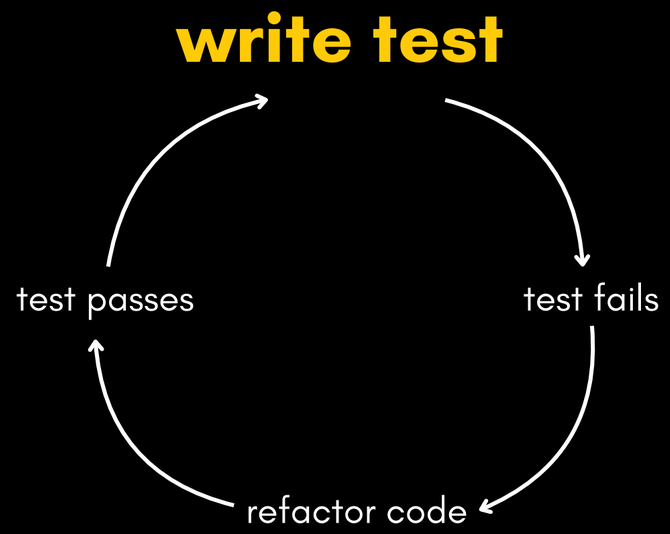


The code



The tests

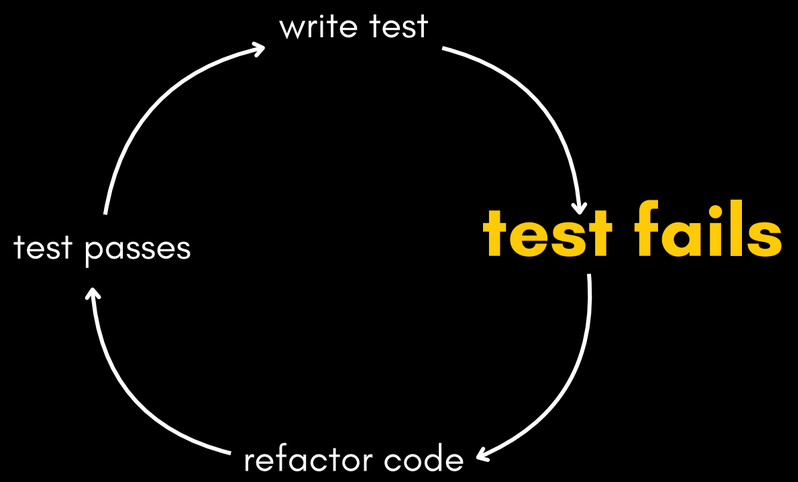
The code



The tests

```
if (is_prime(0) != FALSE) error()
```

The code



The tests

```
if (is_prime(0) != FALSE) error()
```

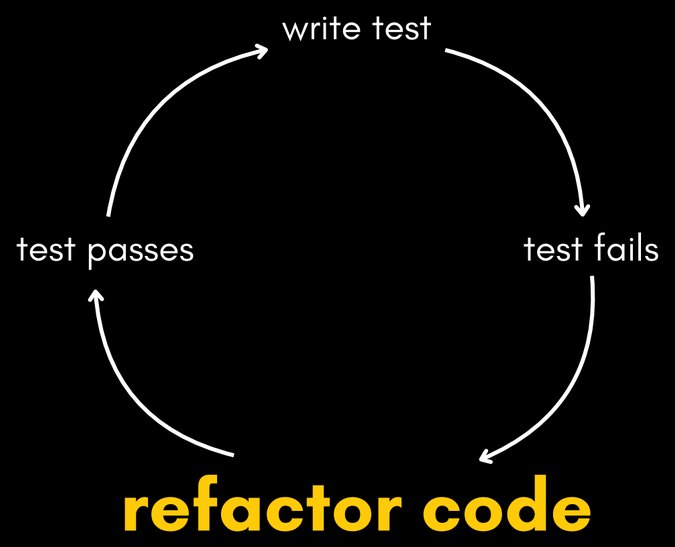


```
# The code
is_prime = function(n) {

    # Special case of zero
    if (n == 0) return(FALSE)

    else return(TRUE)

}
```




```
# The tests
if (is_prime(0) != FALSE) error()
```

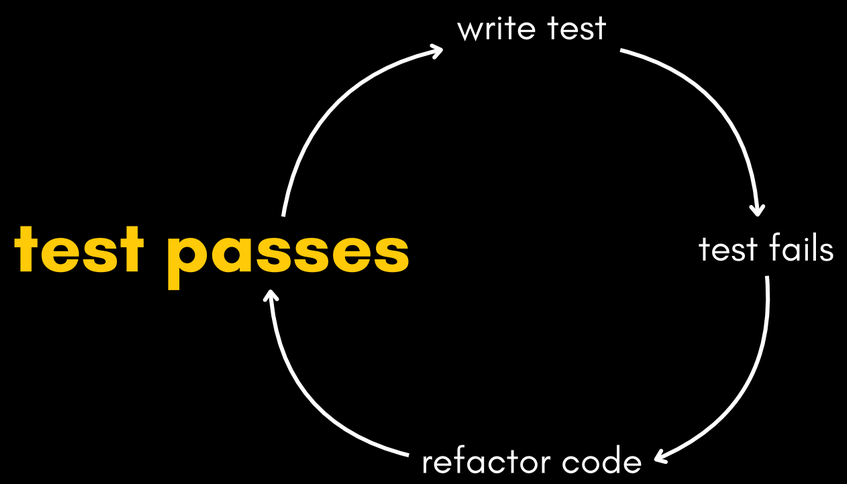


```
# The code
is_prime = function(n) {

    # Special case of zero
    if (n == 0) return(FALSE)

    else return(TRUE)

}
```




```
# The tests
if (is_prime(0) != FALSE) error()
```

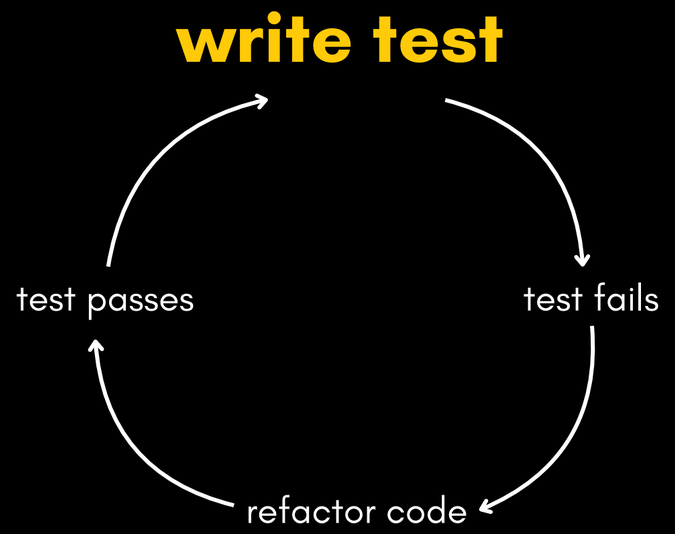


```
# The code
is_prime = function(n) {

    # Special case of zero
    if (n == 0) return(FALSE)

    else return(TRUE)

}
```



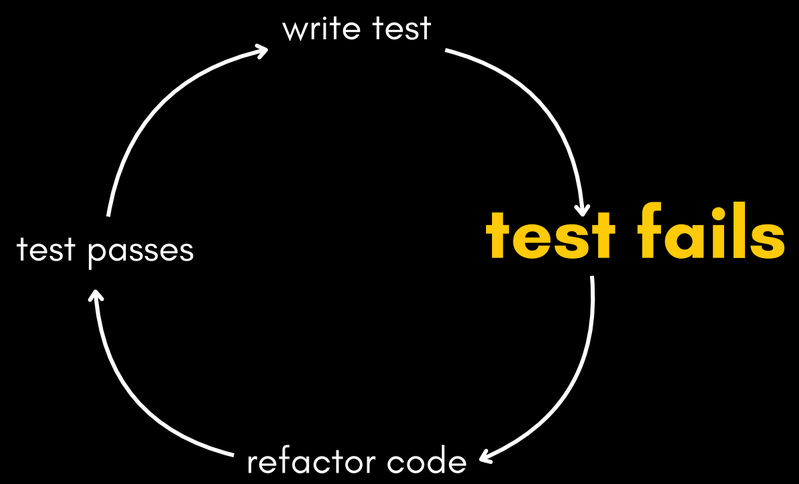

```
# The tests
if (is_prime(0) != FALSE) error()
if (is_prime(1) != TRUE) error()
if (is_prime(2) != TRUE) error()
if (is_prime(4) != FALSE) error()
if (is_prime(5) != TRUE) error()
```

```
# The code
is_prime = function(n) {

    # Special case of zero
    if (n == 0) return(FALSE)

    else return(TRUE)

}
```




```
# The tests
if (is_prime(0) != FALSE) error()
if (is_prime(1) != TRUE) error()
if (is_prime(2) != TRUE) error()
if (is_prime(4) != FALSE) error()
if (is_prime(5) != TRUE) error()
```

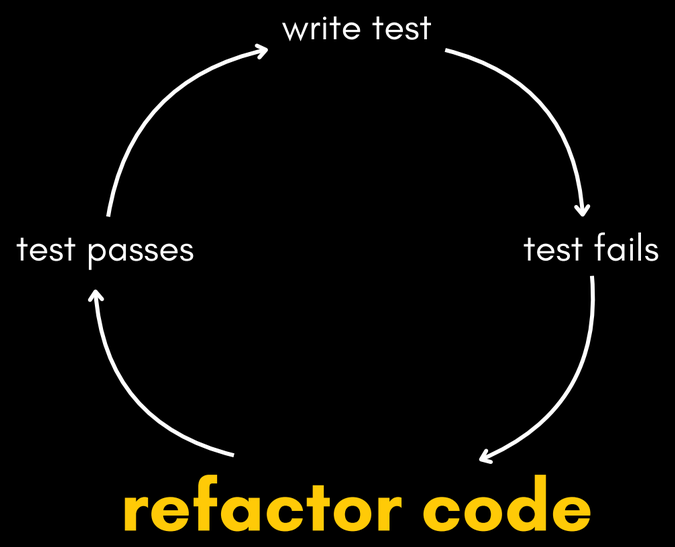


```
# The code
is_prime = function(n) {

    # Special case of zero
    if (n == 0) return(FALSE)

    # ... complicated algorithm

}
```




```
# The tests
if (is_prime(0) != FALSE) error()
if (is_prime(1) != TRUE) error()
if (is_prime(2) != TRUE) error()
if (is_prime(4) != FALSE) error()
if (is_prime(5) != TRUE) error()
```

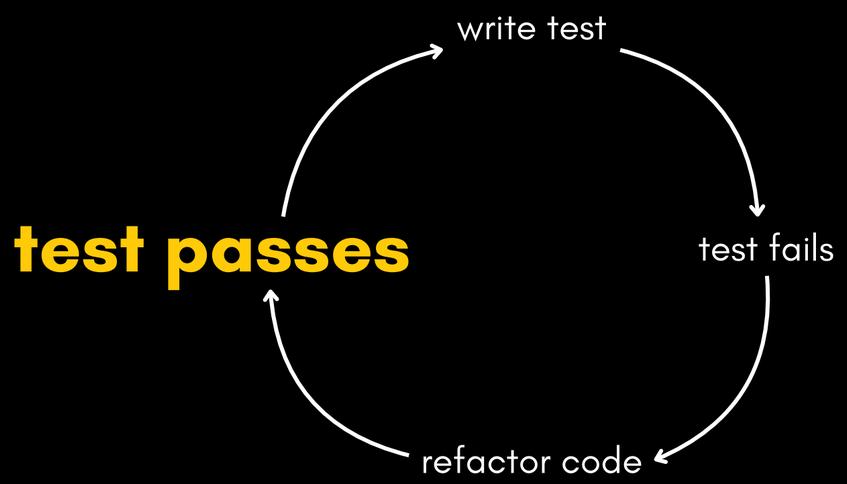


```
# The code
is_prime = function(n) {

   # Special case of zero
   if (n == 0) return(FALSE)

   # ... complicated algorithm

}
```




```
# The tests
if (is_prime(0) != FALSE) error()
if (is_prime(1) != TRUE) error()
if (is_prime(2) != TRUE) error()
if (is_prime(4) != FALSE) error()
if (is_prime(5) != TRUE) error()
```



```python
import random

def is_prime(n, k=10):
    # k = number of rounds (higher = more accuracy)

    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0:
        return False

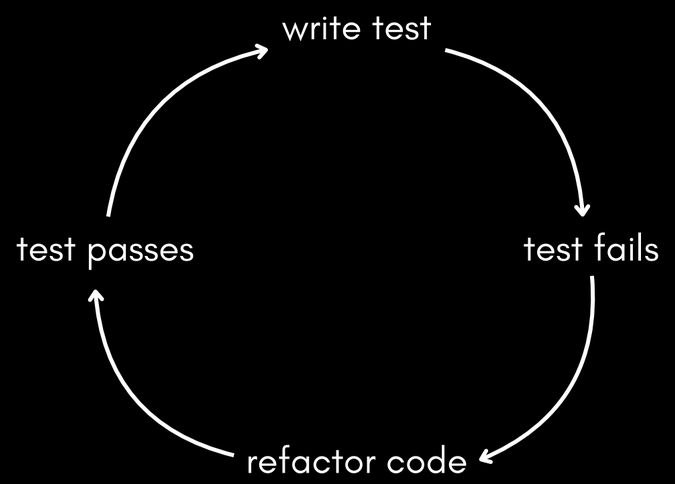
    # write n-1 = d * 2^r
    r = 0
    d = n - 1
    while d % 2 == 0:
        d //= 2
        r += 1

    # witness loop
    for _ in range(k):
        a = random.randrange(2, n - 1)
        x = pow(a, d, n)  # fast modular exponentiation

        if x == 1 or x == n - 1:
            continue

        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False  # definitely composite

    return True  # probably prime
```




```
# The tests
if (is_prime(0) != FALSE) error()
if (is_prime(1) != TRUE) error()
if (is_prime(2) != TRUE) error()
if (is_prime(4) != FALSE) error()
if (is_prime(5) != TRUE) error()
```

Clune (2012) “Test driven development of scientific models”. https://ntrs.nasa.gov/citations/20120013222

Received: 23 March 2022 | Accepted: 19 July 2022

DOI: 10.1111/2041-210X.13961

Methods in Ecology and Evolution | BRITISH ECOLOGICAL SOCIETY

PERSPECTIVE

A call for clean code to effectively communicate science

Alessandro Filazzola[1,2] | CJ Lortie[3,4]

SCIENTIFIC DATA

Amended: Addendum

OPEN

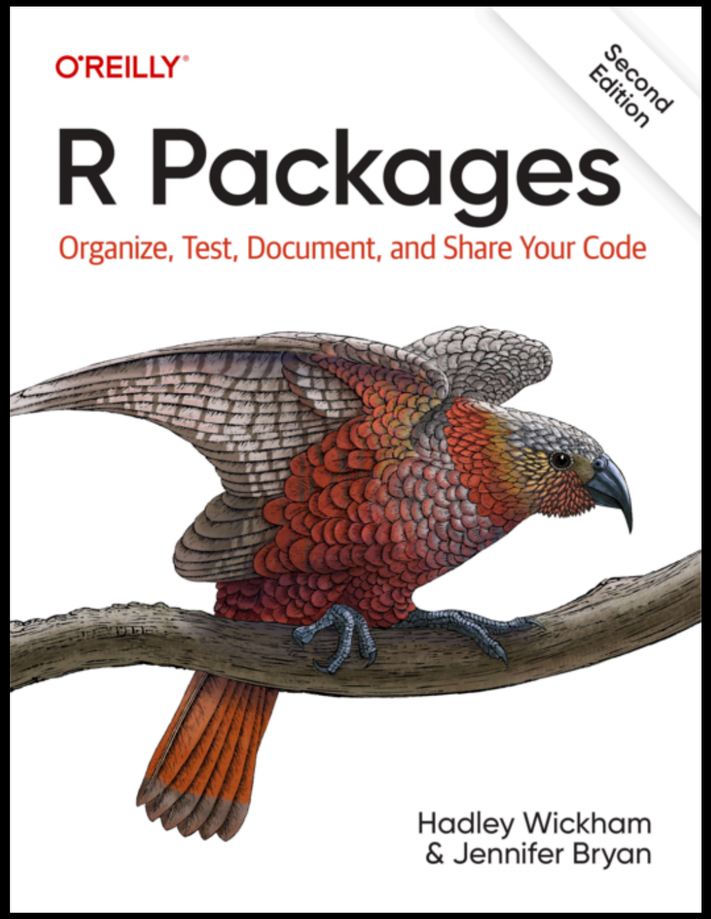
SUBJECT CATEGORIES

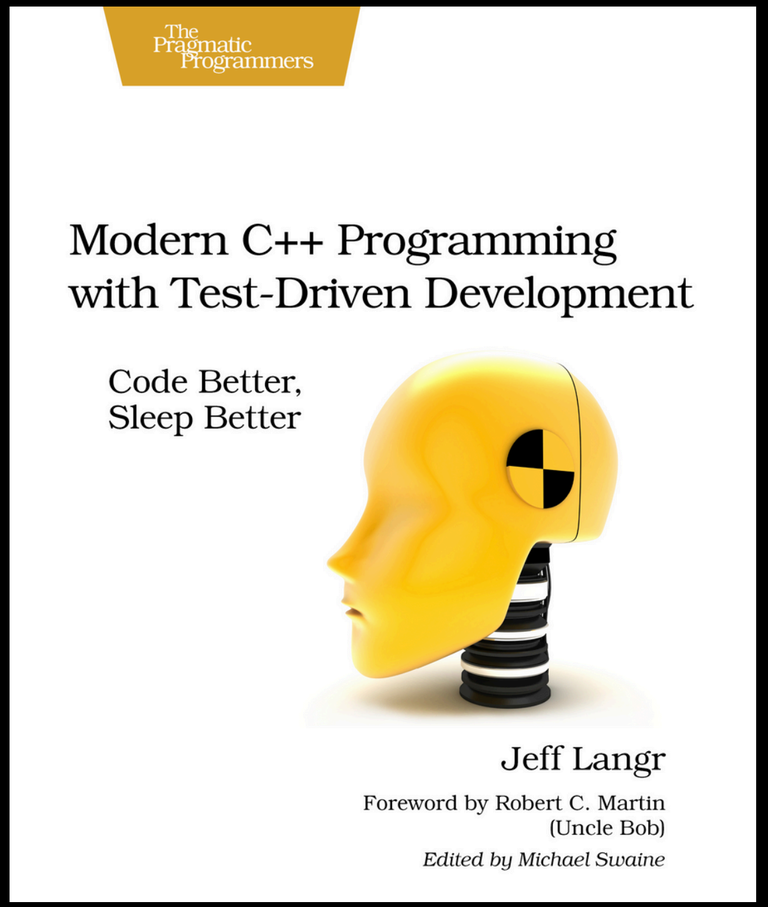
» Research data

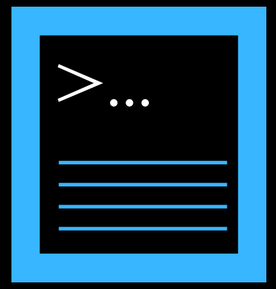
» Publication characteristics

Comment: The FAIR Guiding Principles for scientific data management and stewardship

Mark D. Wilkinson *et al.*[#]


O'REILLY
Second Edition
R Packages
Organize, Test, Document, and Share Your Code
Hadley Wickham
& Jennifer Bryan


The Pragmatic Programmers
Modern C++ Programming with Test-Driven Development
Code Better, Sleep Better
Jeff Langr
Foreword by Robert C. Martin (Uncle Bob)
Edited by Michael Swaine



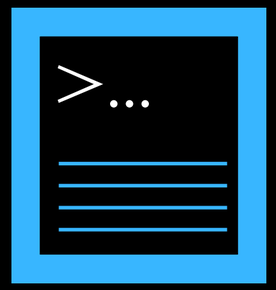
scripts, pipelines, analyses, plotting, stats



simulations, toolboxes, personal libraries



packages, functions, libraries, frameworks



scripts, pipelines, analyses, plotting, stats



simulations, toolboxes, personal libraries



packages, functions, libraries, frameworks

Test your code!



Rampal Etienne



Richel Bilderbeek



Pedro Neves



Sander van Doorn

university of groningen

Beck, K. (2015). Test-driven development: By example (20. printing). Addison-Wesley.
Clune, T. L. (2012). Test driven development of scientific models (No. NTRS Report/Patent Number: GSFC.ABS.6563.2012. NTRS Document ID: 20120013222). NTRS Research Center: Goddard Space Flight Center (GSFC). https://ntrs.nasa.gov/citations/20120013222
Filazzola, A., & Lortie, C. (2022). A call for clean code to effectively communicate science. Methods in Ecology and Evolution, 13(10), 2119–2128. https://doi.org/10.1111/2041-210X.13961
Langr, J. (2014). Modern C++ programming with test-driven development: Code better, sleep better. Pragmatic Programmers.
Netz, C., Ramesh, A., Gismann, J., Gupte, P. R., & Weissing, F. J. (2022). Details matter when modelling the effects of animal personality on the spatial distribution of foragers. Proceedings of the Royal Society B: Biological Sciences, 289(1970), 20210903. https://doi.org/10.1098/rspb.2021.0903
Shi, Y. (2020, January 28). Software reliability in space applications—Facts, trends and challenges. NTRS Meeting Information: Reliability and Maintainability Symposium 2020. https://ntrs.nasa.gov/citations/20200000671
Valente, L., Kristensen, N., Phillimore, A., & Etienne, R. (2021). Report of programming bugs in the DAISIE R package: Consequences and correction. Life Sciences. https://doi.org/10.32942/OSF.IO/W5NTF
Wickham, H. (with Bryan, J.). (2023). R packages (2nd ed). O'Reilly Media, Inc.
Wilkinson, M. D., Dumontier, M., Aalbersberg, Ij. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., da Silva Santos, L. B., Bourne, P. E., Bouwman, J., Brookes, A. J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C. T., Finkers, R., … Mons, B. (2016). The FAIR guiding principles for scientific data management and stewardship. Scientific Data, 3(1), 160018. https://doi.org/10.1038/sdata.2016.18